Nombre:	**Matrícula:**
Carlos Hugo García Gómez	2226690
Alan Bertín Castillo Martínez	2226653
Saúl Alejandro Hernández Garza	2226664
Nombre del curso: Componentes de lógica de negocio	**Nombre del profesor**: Miguel Ángel Gómez Marroquín
Modulo Netacad: 3	**Actividad**: ADRS
Fecha: 14/10/2026	

ADRS

Los Architecture Decision Records (ADRs) documentan las decisiones arquitectónicas más importantes tomadas durante el diseño del sistema EduTech Academy. Cada ADR describe el contexto de la decisión, la solución adoptada, las alternativas consideradas y las consecuencias derivadas de dicha decisión.

ADR-001	**Uso de Domain Model como patrón principal** Reporte
Descripcion	EduTech Academy presenta reglas de negocio complejas relacionadas con certificación, control de cupos, estados del estudiante, pagos, cambios de grupo, modificación de calificaciones, reinscripciones y sustitución de instructor. Además, existen reglas que todavía pueden cambiar, como la asistencia mínima, el plazo de pago, el costo de reinscripción y el límite de cambios de grupo
Decisión	Se utilizará Domain Model como patrón principal para representar la lógica de negocio del sistema. Las reglas serán encapsuladas dentro de entidades, agregados y objetos de valor como Curso, Inscripción, Expediente Académico, Certificado, Cupo y Criterios de Certificación
Alternativas consideradas	• Transaction Script • Table Module • Active Record •
Consecuencias	Las reglas de negocio quedan centralizadas en un solo lugar, se facilita el mantenimiento, las pruebas y la evolución del sistema. Como desventaja, aumenta la complejidad inicial del diseño

ADR-002	**Uso de Service Layer para coordinar casos de uso**
Descripción	Los casos de uso de EduTech Academy requieren coordinar distintos objetos del dominio. Por ejemplo, una inscripción puede involucrar al estudiante, curso, grupo, inscripción y pago
Decisión	Se implementará una Service Layer encargada de coordinar los casos de uso del sistema. Cada Application Service recuperará los objetos necesarios, invocará el comportamiento del dominio, persistirá los cambios y coordinará los eventos generados La Service Layer no contendrá reglas de negocio, ya que estas permanecerán dentro del Domain Model
Alternativas consideradas	Colocar la lógica directamente en controladores o utilizar un Transaction Script independiente para cada operación
Consecuencias	Se logra una separación clara entre coordinación y reglas de negocio. Como consecuencia negativa, se agrega una capa adicional a la arquitectura

ADR-003	**Uso de Repository para acceso a persistencia**
Descripción	Los objetos del dominio deben almacenarse y recuperarse sin depender directamente de la base de datos.
Decisión	Se utilizará el patrón Repository como abstracción para guardar y recuperar agregados y entidades del dominio. La Service Layer utilizará los repositorios para acceder a la persistencia.
Alternativas consideradas	Active Record o acceso directo a la base de datos desde las entidades.
Consecuencias	El dominio queda desacoplado para persistencia y puede probarse con mayor facilidad. Como desventaja, se requiere crear interfaces y clases adicionales para los repositorios

ADR-004	**Uso de Aggregates como límites de consistencia**
Descripción	Existen objetos relacionados que deben mantener reglas consistentes entre sí. Por ejemplo, Curso administra Grupo y Sesión, Inscripción contiene Pago y Expediente Académico contiene Asistencia y Evaluación
Decisión	Se utilizarán Aggregates con un Aggregate Root como punto principal de acceso para realizar modificaciones. Los objetos internos no deberán modificarse directamente desde fuera del agregado
Alternativas consideradas	Permitir modificaciones directas sobre entidades internas o modelar todas las entidades de forma independiente
Consecuencias	Se protege la consistencia del modelo y se reduce el riesgo de cambios inválidos. Como desventaja, se requiere definir adecuadamente los límites de cada agregado

ADR-005	**Uso de Value Objects para conceptos sin identidad**
Descripción	Dentro del dominio existen conceptos que se definen únicamente por sus valores y no requieren una identidad propia
Decisión	Se modelarán como Value Objects conceptos como Cupo, Calificación, Dinero, Modalidad, Calendario del Curso, Criterios de Certificación y Estados
Alternativas consideradas	Representar estos valores únicamente como tipos primitivos o campos simples dentro de las entidades
Consecuencias	Las validaciones quedan encapsuladas dentro de cada concepto, se reduce la repetición de reglas y se mejora la claridad del modelo

ADR-006	**Uso de Domain Events para comunicación entre procesos**
Descripción	Algunas acciones del sistema generan consecuencias en otras partes del negocio. Por ejemplo, cancelar un curso puede generar reembolsos, reinscripciones y registros en la bitácora
Decisión	Se utilizarán Domain Events para representar hechos importantes del negocio y permitir que otros componentes reaccionen sin existir un acoplamiento directo Entre los eventos definidos se encuentran Inscripción Registrada, Pago Registrado, Cupo Agotado, Curso Cancelado, Instructor Sustituido, Calificación Modificada, Estudiante Reprobado, Cambio de Grupo Autorizado, Estudiante Apto para Certificación y Certificado Emitido
Alternativas consideradas	Llamadas directas entre componentes o colocar toda la coordinación dentro de la Service Layer
Consecuencias	Se reduce el acoplamiento entre componentes y se facilita la extensión del sistema. Como desventaja, puede resultar más complejo seguir el flujo completo de una operación

ADR-007	**Bitácora de auditoría inmutable**
Descripción	EduTech Academy necesita mantener trazabilidad de operaciones sensibles como modificaciones de calificaciones, cambios de grupo, sustituciones de instructor y otras acciones importantes
Decisión	Se implementará una Bitácora de Auditoría inmutable. Cada registro almacenará quién realizó la operación, cuándo ocurrió, el valor anterior, el valor nuevo y quién autorizó el cambio, una vez creado el registro, no podrá ser modificado
Alternativas consideradas	Realizar auditoría manual dentro de cada operación o utilizar únicamente triggers en la base de datos
Consecuencias	Se obtiene una trazabilidad confiable de las operaciones importantes y se facilita la identificación de cambios y responsables. Como desventaja, aumenta la cantidad de información almacenada

ADR-008	**Parametrización de reglas de negocio aún no definidas**
Descripción	Algunas reglas del negocio todavía no han sido confirmadas por EduTech Academy, como la asistencia mínima, el plazo de pago, el costo de reinscripción y el límite de cambios de grupo
Decisión	Estas reglas serán implementadas como valores configurables y no como valores fijos dentro del código
Alternativas consideradas	Definir valores temporales como constantes o esperar a que todas las reglas estén definidas antes de desarrollar el sistema
Consecuencias	El sistema podrá adaptarse fácilmente cuando EduTech confirme las políticas definitivas, evitando modificar grandes partes de la lógica de negocio

ADR-009	**Uso de consultas simples para operaciones de solo lectura**
Descripción	No todas las operaciones del sistema requieren utilizar todo el Domain Model. Algunas consultas únicamente leen información y no contienen reglas de negocio complejas Entre estas operaciones se encuentran el catálogo de cursos, el reporte de pagos pendientes y la consulta de la bitácora
Decisión	Para estas operaciones se permitirá utilizar consultas directas o un enfoque simple como Transaction Script, sin cargar todo el Domain Model
Alternativas consideradas	Obligar a que todas las operaciones, incluso las consultas simples, utilicen el Domain Model completo
Consecuencias	Se reduce la complejidad y la sobrecarga en operaciones de solo lectura. Como consecuencia, no todas las operaciones del sistema utilizarán exactamente el mismo estilo arquitectónico